

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK**

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	Shahanaj c	Reg. No.:	192372292
Section / Batch:	CSE AI/2023	Date:	24/07/2026

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
---------------	------------	-----------	-------

Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – DEBUGGING & OPTIMISATION TASK

1Q. Debugging Insertion Sort Code (Shifting Stops One Step Too Early)

The following Insertion Sort implementation stops the shifting process too early because the loop condition excludes index 0 from comparison, causing incorrect placement when the key belongs at the beginning of the array. Carefully analyze the while-loop boundary and explain why the first position is not handled correctly. Apply debugging step-by-step so that all larger elements, including the element at index 0, are shifted when necessary. Optimize the implementation for correctness, clarity, and consistent indexing. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean code with proper comments.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j > 0 and arr[j] > key: # ERROR
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
```

Code of Conduct

I _SHAHANAJ(192372292)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____shahanaj_____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20 %	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25 %	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20 %	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20 %	Demonstrates strong logical reasoning and clear	Shows reasonable analysis with	Limited analysis; weak reasoning	No analytical reasoning demonstrated

		justification of solutions	some justification		
Code Quality	15 %	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1	4	3	3	2	4	39
Q2	4	4	4	3	2	44
Overall Total (100)						83

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Debugging Insertion Sort Code

(a) Problem Analysis

The error is in the while-loop condition:

while $j > 0$ and $\text{arr}[j] > \text{key}$:

Here, $j > 0$ means the loop stops when j becomes 0.

As a result:

- The element at **index 0 is never compared** with key.
- If the key should be inserted at the first position, it is placed incorrectly.

Example

Input:

[5, 2, 4]

Iteration:

- $\text{key} = 2$
- $j = 0$

Condition:

$j > 0$

becomes

$0 > 0 \rightarrow \text{False}$

The loop never executes.

Result:

[5, 2, 4]

Correct output should be:

[2, 5, 4]

Hence, the first element is never shifted.

(b) Step-by-Step Debugging

Original code:

while $j > 0$ and $\text{arr}[j] > \text{key}$:

Step 1

Allow index 0 to be checked.

Replace

$j > 0$

with

$j \geq 0$

Now the loop becomes

while $j \geq 0$ and $\text{arr}[j] > \text{key}$:

Step 2

Shift all larger elements one position to the right.

$\text{arr}[j+1] = \text{arr}[j]$

Step 3

Decrease the index.

$j -= 1$

Step 4

Insert the key.

$\text{arr}[j+1] = \text{key}$

Now insertion works correctly even when the key belongs at index 0.

(c) Corrected Code

```
def insertion_sort(arr):
```

```
    # Traverse from the second element
```

```
    for i in range(1, len(arr)):
```

```
        key = arr[i]      # Element to insert
```

```
        j = i - 1
```

```
        # Shift all elements greater than key
```

```
while j >= 0 and arr[j] > key:
```

```
    arr[j + 1] = arr[j]
```

```
    j -= 1
```

```
# Insert key at its correct position
```

```
arr[j + 1] = key
```

```
return arr
```

```
# Example
```

```
arr = [5, 2, 4, 6, 1, 3]
```

```
print(insertion_sort(arr))
```

Output

```
[1, 2, 3, 4, 5, 6]
```

(d) Time Complexity

Case	Complexity
Best Case (already sorted)	$O(n)$
Average Case	$O(n^2)$
Worst Case (reverse sorted)	$O(n^2)$
Space Complexity	$O(1)$

